



Netflix Content Analysis

Using SQL

Data Analyst Portfolio Project

Prepared by: **Karandeep Sandul**

Tool: MySQL | Dataset: Netflix Titles (8,807 records)

2024

Table of Contents

- 1. Project Overview**
- 2. Dataset Information**
- 3. Data Cleaning**
- 4. Key Findings at a Glance**
- 5. Detailed Analysis & SQL Queries**
 - 5.1 Content Distribution — Movies vs TV Shows
 - 5.2 Content Ratings Analysis
 - 5.3 Country-Level Analysis
 - 5.4 Director Analysis
 - 5.5 Genre Analysis (Recursive CTE)
 - 5.6 Actor / Cast Analysis
 - 5.7 Movie Duration Analysis
 - 5.8 Time-Based Analysis
- 6. SQL Concepts Practiced**
- 7. Challenges Faced**
- 8. Conclusion & Future Plans**

1. Project Overview

I built this project to practise SQL using a real-world dataset. Netflix is a platform most people are familiar with, so I thought it would be interesting to explore what their content library actually looks like through data. The goal was not just to write queries, but to answer questions that a business analyst might genuinely need to answer—how content is distributed, which countries contribute the most, how ratings break down, and whether Netflix has been adding content faster in recent years.

The project covers 35+ SQL queries across a wide range of concepts, from basic filtering and aggregation to recursive CTEs, window functions, and string manipulation on multi-valued columns like genres and cast.

Objectives

- Explore the Netflix dataset and understand its structure.
- Clean the data before analysis (NULLs, date formatting, type casting).
- Answer 35+ business and analytical questions using SQL.
- Practice advanced SQL including CTEs, window functions, and recursive queries.
- Document findings clearly for a portfolio audience.

Tools Used

Tool	Purpose
MySQL	Query writing and analysis
MySQL Workbench	Running and testing queries
GitHub	Version control and portfolio hosting
Microsoft Excel	Initial dataset viewing

2. Dataset Information

The dataset used for this project is the publicly available Netflix Movies and TV Shows dataset. It contains information about titles available on Netflix up to mid-2021.

Column	Description	Example
show_id	Unique ID for each title	s1
type	Movie or TV Show	Movie
title	Name of the title	Kota Factory
director	Director(s) of the title	Rajiv Chilaka
cast	Main cast members (comma-separated)	Anupam Kher, Shah Rukh Khan
country	Country of production	United States
date_added	Date added to Netflix	September 25, 2021
release_year	Original release year	2021
rating	Content rating	TV-MA
duration	Length (min for movies, seasons for TV)	99 min / 2 Seasons
listed_in	Genre tags (comma-separated)	Dramas, International Movies
description	Brief synopsis	Text description

8,807 Total Titles	6,131 Movies	2,676 TV Shows	12 Columns
------------------------------	------------------------	--------------------------	----------------------

3. Data Cleaning

Before starting any analysis I had to deal with a few data quality issues. The dataset is fairly clean but has quirks that show up quickly once you start querying.

NULL values in key columns

The director and cast columns have many NULLs — these rows are filtered using WHERE director IS NOT NULL before any director-level aggregation, so they don't distort counts.

Duration stored as text

Duration is stored as a string like '99 min' or '2 Seasons'. I used REPLACE() combined with CAST(... AS UNSIGNED) to extract numeric values for comparisons and averages.

Date stored as a formatted string

The date_added column holds values like 'September 25, 2021'. I used STR_TO_DATE(date_added, '%M %d, %Y') to convert these into proper DATE values so I could extract year and month with YEAR() and MONTHNAME().

Comma-separated multi-values

Both listed_in (genres) and cast contain multiple values in a single cell. I used Recursive CTEs to split these into individual rows for accurate counting.

Duplicate-looking director query

An initial query counted directors without filtering NULLs, which inflated the 'NULL director' group. A revised query adds WHERE director IS NOT NULL to fix this.

4. Key Findings at a Glance

#	Finding	Figure
1	Netflix library is movie-heavy	69.62% Movies / 30.38% TV Shows
2	Most common rating (Movies)	TV-MA (2,062 titles)
3	Most common rating (TV Shows)	TV-MA (1,145 titles)
4	Top content-producing country	United States — 2,818 titles
5	Most prolific director on Netflix	Rajiv Chilaka — 19 titles
6	Average movie duration	99.58 minutes
7	Longest movie	Black Mirror: Bandersnatch (312 min)
8	Shortest movie	Silent (3 min)
9	Most common genre	International Movies — 2,752 titles
10	Most frequently appearing actor	Anupam Kher — 43 titles
11	Peak content-addition year	2019 — 1,999 titles added
12	Month with most additions	July — 819 titles
13	Avg delay (release → Netflix)	4.69 years
14	Shah Rukh Khan titles on Netflix	35 titles
15	Titles with 'Love' in the title	188 titles

5. Detailed Analysis & SQL Queries

This section walks through the most interesting queries from the project. I have selected queries that demonstrate different SQL techniques — from basic aggregation to window functions and recursive CTEs. All numbers are from running the queries against the actual dataset.

5.1 Content Distribution — Movies vs TV Shows

The first question I wanted to answer was the basic split of the library. Rather than just counting each type, I calculated percentages to make the finding meaningful.

```
SELECT
  type,
  COUNT(*) AS total_titles,
  ROUND(
    COUNT(*) * 100.0 / (SELECT COUNT(*) FROM netflix_data),
    2
  ) AS percentage
FROM netflix_data
GROUP BY type;
```

Type	Total Titles	Percentage
Movie	6,131	69.62%
TV Show	2,676	30.38%

■ Insight:

Almost 7 in 10 titles on Netflix are movies. This tells us Netflix built its library around films first and expanded into original TV content later — consistent with what we know historically about the platform.

5.2 Content Ratings Analysis

I ran two separate queries to find the most common rating for Movies and for TV Shows independently.

```
-- Most common rating for Movies
SELECT rating, COUNT(*) AS rating_count
FROM netflix_data
WHERE type = 'Movie'
GROUP BY rating
ORDER BY rating_count DESC
LIMIT 1;

-- Most common rating for TV Shows
SELECT rating, COUNT(*) AS rating_count
FROM netflix_data
WHERE type = 'TV Show'
GROUP BY rating
ORDER BY rating_count DESC
LIMIT 1;
```

Content Type	Most Common Rating	Count
Movie	TV-MA	2,062

Content Type	Most Common Rating	Count
TV Show	TV-MA	1,145

Full rating distribution:

Rating	Count
TV-MA	3,207
TV-14	2,160
TV-PG	863
R	799
PG-13	490
TV-Y7	334
TV-Y	307
PG	287
TV-G	220

■ **Insight:**

TV-MA is the dominant rating across both categories, suggesting Netflix primarily targets adult audiences. TV-Y and TV-G content exists but is a smaller portion — Netflix Kids is a separate experience for a reason.

5.3 Country-Level Analysis

I used a window function — `DENSE_RANK()` — to rank countries by the number of titles they contribute. This is more useful than a simple `ORDER BY` because it handles ties cleanly.

```
SELECT
    country,
    COUNT(*) AS total_titles,
    DENSE_RANK() OVER (ORDER BY COUNT(*) DESC) AS country_rank
FROM netflix_data
WHERE country IS NOT NULL
GROUP BY country;
```

Rank	Country	Total Titles
1	United States	2,818
2	India	972
3	United Kingdom	419
4	Japan	245
5	South Korea	199
6	Canada	181
7	Spain	145
8	France	124
9	Mexico	110
10	Egypt	106

■ Insight:

The United States leads by a large margin with 2,818 titles — nearly 3x more than India in second place. India's strong position reflects Netflix's heavy investment in Bollywood and regional Indian content. South Korea's appearance in the top five is notable given the global rise of K-dramas.

5.4 Director Analysis

Two director queries stood out. First, the top 10 most prolific directors. Second, a query to find directors who have directed both Movies and TV Shows — which required using `SUM()` with a boolean condition inside `GROUP BY`.

```
-- Top 10 directors by title count
SELECT director, COUNT(*) AS total_titles
FROM netflix_data
WHERE director IS NOT NULL
GROUP BY director
ORDER BY total_titles DESC
LIMIT 10;

-- Directors with both Movies and TV Shows
SELECT
    director,
    SUM(type = 'Movie') AS Movies,
```

```

SUM(type = 'TV Show') AS TV_Shows
FROM netflix_data
WHERE director IS NOT NULL
GROUP BY director
HAVING Movies > 0 AND TV_Shows > 0;

```

Director	Total Titles
Rajiv Chilaka	19
Raúl Campos, Jan Suter	18
Suhas Kadav	16
Marcus Raboy	16
Jay Karas	14
Cathy Garcia-Molina	13
Youssef Chahine	12
Martin Scorsese	12
Jay Chapman	12
Steven Spielberg	11

■ **Insight:**

Rajiv Chilaka, known for animated Indian children's content, tops the list. The presence of Martin Scorsese and Steven Spielberg shows that Netflix has a strong back-catalogue of Hollywood classics. 48 directors have worked in both Movies and TV Show formats on the platform.

5.5 Genre Analysis — Recursive CTE

The `listed_in` column stores genres as a comma-separated string like 'Dramas, International Movies, Romantic Movies'. To count titles per genre accurately I had to split this column into individual rows. I used a Recursive CTE to do this — the most technically challenging query in the project.

```
WITH RECURSIVE genre_split AS (  
  -- Anchor: extract first genre from each title  
  SELECT  
    title,  
    TRIM(SUBSTRING_INDEX(listed_in, ',', 1)) AS genre,  
    SUBSTRING(  
      listed_in,  
      LENGTH(SUBSTRING_INDEX(listed_in, ',', 1)) + 2  
    ) AS remaining_genre  
  FROM netflix_data  
  
  UNION ALL  
  
  -- Recursive: extract next genre until none remain  
  SELECT  
    title,  
    TRIM(SUBSTRING_INDEX(remaining_genre, ',', 1)),  
    SUBSTRING(  
      remaining_genre,  
      LENGTH(SUBSTRING_INDEX(remaining_genre, ',', 1)) + 2  
    )  
  FROM genre_split  
  WHERE remaining_genre <> ''  
)  
SELECT genre, COUNT(*) AS total_titles  
FROM genre_split  
GROUP BY genre  
ORDER BY total_titles DESC;
```

Genre	Total Titles
International Movies	2,752
Dramas	2,427
Comedies	1,674
International TV Shows	1,351
Documentaries	869
Action & Adventure	859
TV Dramas	763
Independent Movies	756
Children & Family Movies	641
Romantic Movies	616

■ **Insight:**

International Movies is the single largest genre category, which makes sense given Netflix's global expansion. Dramas are the most popular content type by genre. The strong showing of Documentaries and Independent Movies shows Netflix serves niche audiences alongside mainstream content.

5.6 Actor / Cast Analysis — Recursive CTE

Similar to the genre split, the cast column stores multiple actors per row. I applied the same Recursive CTE approach to find the most frequently appearing actors across all titles.

Actor	Total Titles
Anupam Kher	43
Shah Rukh Khan	35
Julie Tejwani	33
Naseeruddin Shah	32
Takahiro Sakurai	32
Rupa Bhimani	31
Akshay Kumar	30
Om Puri	30
Yuki Kaji	29
Amitabh Bachchan	28

■ Insight:

Anupam Kher appears in 43 Netflix titles — more than any other actor on the platform. 8 of the top 10 most-appearing actors are from India's film industry, reflecting how heavily Indian Bollywood content features in Netflix's catalogue. I also wrote a separate query to check that Shah Rukh Khan specifically appears in 35 titles.

5.7 Movie Duration Analysis

Duration required string cleaning before any numeric analysis could be done. I used REPLACE() to strip the ' min' suffix and CAST(...AS UNSIGNED) to convert to a number.

```
-- Average movie duration
SELECT ROUND(AVG(CAST(REPLACE(duration, ' min', '') AS UNSIGNED)), 2)
      AS average_movie_duration
FROM netflix_data
WHERE type = 'Movie';

-- Longest movie (subquery to match exact max)
SELECT title, duration
FROM netflix_data
WHERE type = 'Movie'
AND CAST(REPLACE(duration, ' min', '') AS UNSIGNED) =
      (SELECT MAX(CAST(REPLACE(duration, ' min', '') AS UNSIGNED))
       FROM netflix_data WHERE type = 'Movie');
```

Metric	Result
Average movie duration	99.58 minutes (~1 hr 40 min)
Longest movie	Black Mirror: Bandersnatch — 312 min
Shortest movie	Silent — 3 min
Movies longer than 120 min	1,142 titles
TV Shows with more than 3 seasons	259 shows

■ Insight:

The average Netflix movie runs just under 100 minutes, which aligns with typical theatrical film lengths. Black Mirror: Bandersnatch at 312 minutes is an outlier — it is an interactive film where total runtime depends on viewer choices, making it technically much longer than a standard movie.

5.8 Time-Based Analysis

This section required converting the date_added string column into an actual date using STR_TO_DATE(). Once converted I could extract year and month to find patterns in how Netflix grows its catalogue.

```
-- Titles added each year
SELECT
      YEAR(STR_TO_DATE(date_added, '%M %d, %Y')) AS year_added,
      COUNT(*) AS total_titles
FROM netflix_data
WHERE date_added IS NOT NULL
GROUP BY year_added
ORDER BY year_added DESC;

-- Average delay between release and Netflix addition
SELECT
      ROUND(
        AVG(YEAR(STR_TO_DATE(date_added, '%M %d, %Y')) - release_year),
        2
      ) AS average_delay_years
```

```
FROM netflix_data
WHERE date_added IS NOT NULL;
```

Year Added	Titles Added
2021	1,498
2020	1,878
2019	1,999 (peak)
2018	1,625
2017	1,164
2016	418
2015	73
2014	23
2013	10
2012	3

Metric	Result
Peak year for additions	2019 — 1,999 titles
Month with most additions	July — 819 titles
Titles added in September	765 titles
Avg delay (release → Netflix)	4.69 years
Titles added within 1 year of release	4,793 titles
Titles added >5 years after release	2,105 titles

■ Insight:

2019 was Netflix's peak content-addition year with nearly 2,000 titles. Growth accelerated dramatically from 2016 onward — only 418 titles were added in 2016 versus 1,999 three years later. On average, content reaches Netflix about 4.69 years after its original release, though over half of all titles were added within a year of release, showing Netflix is actively acquiring both new and back-catalogue content.

6. SQL Concepts Practiced

Category	Concepts Used
Filtering & Sorting	WHERE, LIKE, ORDER BY, LIMIT
Aggregation	COUNT(), AVG(), MIN(), MAX(), ROUND()
Grouping	GROUP BY, HAVING
Conditional Logic	CASE WHEN ... THEN ... END
Subqueries	Scalar subqueries inside WHERE and SELECT
CTEs	WITH ... AS — non-recursive and recursive
Window Functions	DENSE_RANK() OVER (ORDER BY ...)
String Functions	REPLACE(), TRIM(), SUBSTRING_INDEX(), SUBSTRING(), LENGTH()
Date Functions	STR_TO_DATE(), YEAR(), MONTHNAME(), MONTH()
Type Casting	CAST(... AS UNSIGNED) for numeric extraction
Boolean Aggregation	SUM(type = 'Movie') to count conditionally

7. Challenges Faced

Splitting comma-separated columns

The `listed_in` and `cast` columns store multiple values in a single cell. Handling this with a standard query is impossible — you get genre counts per row, not per individual genre. Learning Recursive CTEs to solve this was the biggest technical challenge of the project and took several attempts to get right.

Casting duration to a number

Duration is stored as '99 min' or '2 Seasons'. I had to strip the text suffix with `REPLACE()` before `CAST()` could work. Getting this wrong produced NULL values silently, which I only noticed when the `AVG()` result looked wrong.

Converting date_added to a proper date

MySQL does not automatically recognise 'September 25, 2021' as a date. I had to figure out the correct format string '%M %d, %Y' for `STR_TO_DATE()`. Rows with NULL or blank `date_added` also had to be filtered out to avoid errors.

NULL handling in director/cast queries

Many titles have no director listed. An early version of my director count query included a large 'NULL' group at the top. I fixed it by always adding `WHERE director IS NOT NULL`.

Understanding window functions

`DENSE_RANK()` was new to me before this project. Understanding the difference between `RANK()`, `DENSE_RANK()`, and `ROW_NUMBER()` took some reading, and deciding when to use `OVER()` vs a simple `ORDER BY` required thinking through what each actually produces.

8. Conclusion & Future Plans

This project gave me hands-on experience with SQL at a depth that tutorial exercises do not. Working with a real dataset means dealing with messy text columns, NULLs in unexpected places, and having to think about what the data actually represents before writing any query.

By the end I had written 35+ queries covering the full range of SQL — from basic SELECT and GROUP BY to Recursive CTEs for parsing comma-separated columns, DENSE_RANK() for ranking, and subqueries for finding extremes. More importantly I learned to read the results critically and ask whether an unexpected number is a real insight or a data quality issue.

What I Learned

- Always inspect NULLs and data types before writing analysis queries.
- Recursive CTEs are the right tool for splitting multi-valued columns.
- Window functions like DENSE_RANK() are more expressive than ORDER BY + LIMIT for ranking.
- Real insights require thinking about why a number is what it is, not just reporting it.
- Writing a clean, readable SQL file with comments is as important as getting the right answer.

Future Plans

- Build an interactive Power BI dashboard visualising the key insights from this analysis.
- Perform deeper trend analysis — how has Netflix's content mix changed year by year?
- Compare content quality (rating distribution) across countries.
- Extend the project using Python and Pandas for analysis not easily done in SQL.
- Add stored procedures and views to make the queries reusable.

The complete SQL script and dataset are available on my GitHub repository.

Prepared by **Karandeep Sandul** | Data Analyst Portfolio Project